

SOFTWARE ARCHITECTURE DIAGRAMS GUIDE

Practical Reference for Logistics Platform Design

Comprehensive guide to C4 model diagrams, database design, deployment architecture, and operational workflow visualization for enterprise logistics systems.

Document Version	Final v1.0
Classification	Technical Reference
Date	June 2026
Status	Engineering Team Reference

Introduction

This guide provides a practical reference for the most relevant architecture diagrams when designing a logistics coordination platform and other enterprise SaaS systems. Each diagram type serves a specific purpose in the software development lifecycle, from stakeholder communication to implementation planning and DevOps deployment.

Diagram Organization:

Diagrams are organized into three categories based on project phase:

- **Foundation (Phases 1-5):** Business process, context, user flow, database, and container diagrams — essential for project initiation and design
- **Implementation (Phases 6-8):** Component, sequence, and deployment diagrams — critical for development and DevOps planning
- **Scaling (Phases 9-12):** Data flow, event architecture, network, and security diagrams — necessary for production scaling and enterprise hardening

C4 Model Foundation: The first four diagram types follow the C4 Model (Context, Containers, Components, Code) developed by Simon Brown — a hierarchical approach to visualizing software architecture at different levels of abstraction.

1. System Context Diagram (C4 Level 1)

Purpose: Shows the system as a black box and its relationships with external actors and systems. Used for stakeholder communication and scope definition.

1.1 Diagram Structure

The System Context Diagram places the logistics platform at the center and illustrates all external entities that interact with it:

- **Logistics Manager:** Primary user — monitors operations, assigns dispatches, reviews analytics via web dashboard
- **Dispatcher:** Operational user — coordinates daily deliveries, matches trucks to loads, resolves exceptions
- **Partner Admin:** External partner user — manages trucking fleet, registers drivers, updates fleet status
- **Driver:** Mobile user — receives jobs, updates trip progress, uploads proof of delivery via mobile app
- **Super Admin:** Platform administrator — manages companies, subscriptions, global settings
- **Google Maps API:** External service — provides geocoding, routing, distance matrix, and map tile services
- **SMS Gateway (Twilio):** External service — delivers SMS notifications for driver alerts and OTP verification
- **Email Service (SendGrid):** External service — sends transactional emails, reports, and marketing communications
- **Push Notification Service (Firebase):** External service — delivers push notifications to iOS and Android driver apps
- **ERP System:** External integration — synchronizes inventory, orders, and invoicing data with customer ERP
- **GPS Hardware Provider:** External data source — receives GPS coordinate streams from dedicated vehicle trackers

1.2 Example Diagram Description

Central Node: "Logistics Coordination Platform" (blue box)

User Actors: Logistics Manager, Dispatcher, Partner Admin, Driver, Super Admin (person icons, left and right)

External Systems: Google Maps, Twilio, SendGrid, Firebase, ERP System, GPS Provider (cylinder/database icons, bottom)

Relationships: Labeled arrows showing data direction and interaction type (e.g., "requests dispatch")

assignment", "receives GPS stream")

Best Practice: Keep this diagram technology-agnostic. Show 'what' interacts with the system, not 'how'. Avoid internal component details. Target audience: non-technical stakeholders, executives, and product managers.

2. Container Diagram (C4 Level 2)

Purpose: Decomposes the system into high-level containers (applications, data stores) and shows how they communicate. Used for technical design discussions and technology selection.

2.1 Container Inventory

The logistics platform decomposes into the following containers:

Web Dashboard (Next.js)

Technology: React, Next.js 14, Tailwind CSS, TanStack Query

Single Page Application served via CDN. Logistics managers and dispatchers interact with operational views, analytics, and dispatch workflows.

Partner Portal (Next.js)

Technology: React, Next.js 14, Tailwind CSS

Separate SPA for partner admins to manage fleets, view assignments, and update statuses. Shared component library with Web Dashboard.

Admin Panel (Next.js)

Technology: React, Next.js 14, Tailwind CSS

Internal administration interface for Super Admins. Company management, subscription billing, global configuration.

Driver Mobile App (React Native)

Technology: React Native, Expo, Socket.IO Client

Cross-platform iOS/Android application. Job receipt, GPS streaming, status updates, proof of delivery capture.

API Server (Node.js/NestJS)

Technology: Node.js, NestJS, TypeScript, Express

Primary backend application exposing REST and GraphQL APIs. Handles authentication, business logic, and data persistence.

Real-Time Gateway (Socket.IO)

Technology: Node.js, Socket.IO, Redis

Dedicated WebSocket server handling bidirectional event streaming. Horizontally scaled with Redis Pub/Sub for broadcast.

Analytics Engine (Python)

Technology: Python, Pandas, scikit-learn, FastAPI

Background service processing operational events into analytics aggregates. Scheduled reports and ML inference.

Notification Service (Node.js)

Technology: Node.js, Bull Queue, Twilio SDK

Queue consumer processing notification jobs. Routes to appropriate channel (push, SMS, email) based on user preferences.

Operational Database (PostgreSQL)

Technology: PostgreSQL 15, PostGIS, TimescaleDB

Primary relational database storing companies, trucks, drivers, dispatches, users, and permissions. Multi-tenant via row-level security.

Analytics Database (ClickHouse)

Technology: ClickHouse, Apache Superset

Columnar data warehouse for time-series analytics, GPS logs, and operational metrics. Optimized for aggregation queries.

Cache Layer (Redis)

Technology: Redis Cluster, 6 nodes

In-memory store for session management, API response caching, rate limiting counters, and real-time pub/sub.

Object Storage (S3/MinIO)

Technology: AWS S3 / MinIO, CloudFront CDN

Blob storage for proof of delivery images, documents, backup archives, and static assets.

2.2 Inter-Container Communication

Web Dashboard → **API Server:** HTTPS/REST + GraphQL for data fetching. JWT Bearer tokens for authentication.

Driver App → **Real-Time Gateway:** WebSocket (Socket.IO) for GPS streaming and status updates. Fallback to HTTPS polling.

API Server → **Operational Database:** PostgreSQL wire protocol via connection pool (PgBouncer).

API Server → **Cache Layer:** Redis protocol for caching, sessions, and distributed locks.

API Server → **Analytics Database:** HTTP/ClickHouse native protocol for analytical queries.

API Server → **Notification Service:** Internal message queue (Bull/Redis) for async job dispatch.

Real-Time Gateway → **Cache Layer:** Redis Pub/Sub for cross-server event broadcasting.

Analytics Engine → **Operational Database:** Read replica access for ETL pipeline.

All Containers → **Object Storage:** S3-compatible API for file upload/download.

Best Practice: Each container should be independently deployable and scalable. Container boundaries align with team ownership boundaries in a microservices transition.

3. Component Diagram (C4 Level 3)

Purpose: Decomposes a single container into its internal components (classes, modules, services) and their interactions. Used for code organization and detailed design.

3.1 API Server Components

The API Server container decomposes into the following logical components:

- **Auth Component:** JWT token generation/validation, password hashing (bcrypt), SSO integration (OAuth 2.0/SAML), session management, and MFA workflows.
- **Fleet Component:** Truck CRUD operations, fleet status management, capacity tracking, maintenance scheduling, and partner fleet aggregation.
- **Dispatch Component:** Dispatch lifecycle management, assignment logic, status transition validation, conflict detection, and priority queuing.
- **Tracking Component:** GPS coordinate ingestion, geofencing logic, route deviation detection, ETA calculation, and historical path reconstruction.
- **Analytics Component:** KPI calculation, report generation, dashboard data aggregation, and ML feature extraction.
- **Notification Component:** Multi-channel notification routing, template management, delivery tracking, and preference management.
- **Partner Component:** Partner onboarding, verification workflows, performance scoring, contract management, and quota enforcement.
- **User Component:** User profile management, RBAC permission resolution, role assignment, and audit logging.
- **Integration Component:** ERP adapter framework, webhook management, API key provisioning, and third-party service orchestration.
- **Middleware Layer:** Request validation, rate limiting, tenant resolution, logging, error handling, and request tracing.

3.2 Component Interaction Patterns

Request Flow: HTTP Request → Middleware Layer (auth, rate limit, tenant) → Router → Component → Repository → Database

Event Flow: Component emits domain event → Event Bus (Kafka/Redis) → Subscribing Components process asynchronously

Cross-Cutting: Logging, metrics, and tracing applied via middleware and decorators across all components

Best Practice: Components should align with domain boundaries (Domain-Driven Design). Each component has its own repository interface, service layer, and DTO definitions. Avoid circular dependencies between components.

4. Database Entity-Relationship Diagram

Purpose: Shows database entities, their attributes, and relationships. Critical for data modeling, query optimization, and team alignment on data structures.

4.1 Core Entity Relationships

The logistics platform database centers around these primary entities:

- **Company:** Root tenant entity. All other entities belong to exactly one company. Fields: id, name, subscription_tier, created_at, settings (JSONB).
- **Partner:** Trucking partner organization linked to one or more companies. Fields: id, company_id, name, contact_info, reliability_score, contract_status.
- **Truck:** Vehicle asset owned by a partner. Fields: id, partner_id, company_id, plate_number, type, capacity, status, gps_coords, last_updated.
- **Driver:** Individual driver profile linked to partner and optionally assigned truck. Fields: id, partner_id, user_id, license_number, rating, status.
- **Dispatch:** Delivery assignment linking truck, driver, route, and cargo. Fields: id, company_id, truck_id, driver_id, origin, destination, status, timeline.
- **Delivery:** Individual delivery leg within a dispatch. Fields: id, dispatch_id, waypoint_index, location, status, proof_of_delivery.
- **Route:** Predefined or computed route definition. Fields: id, company_id, name, waypoints (JSONB), distance_km, estimated_duration.
- **TrackingLog:** Time-series GPS and status event. Fields: id, truck_id, dispatch_id, latitude, longitude, accuracy, speed, timestamp.
- **MaintenanceRecord:** Scheduled or ad-hoc maintenance event. Fields: id, truck_id, type, description, scheduled_date, completed_date, cost.
- **Notification:** Multi-channel notification record. Fields: id, user_id, type, channel, content, status, sent_at, read_at.
- **User:** Platform user account. Fields: id, company_id, email, password_hash, role, status, last_login.
- **Permission:** Granular permission definition. Fields: id, resource, action, description.
- **RolePermission:** Many-to-many join between roles and permissions.
- **AuditLog:** Immutable compliance record. Fields: id, company_id, user_id, action, entity_type, entity_id, changes (JSONB), timestamp.

4.2 Relationship Cardinality

Company → Partner: One-to-Many (one company has multiple partner relationships)

Company → Truck: One-to-Many via Partner (trucks belong to partners, partners belong to companies)

Partner → Truck: One-to-Many (one partner owns multiple trucks)

Partner → Driver: One-to-Many (one partner employs multiple drivers)

Truck → Dispatch: One-to-Many (one truck handles multiple dispatches over time)

Driver → Dispatch: One-to-Many (one driver handles multiple dispatches)

Dispatch → Delivery: One-to-Many (one dispatch has multiple delivery legs/waypoints)

Truck → TrackingLog: One-to-Many (time-series GPS records per truck)

Truck → MaintenanceRecord: One-to-Many (maintenance history per truck)

User → Notification: One-to-Many (notifications per user)

Company → User: One-to-Many (users per tenant company)

Best Practice: Use crow's foot notation for cardinality. Include foreign key constraints and cascade rules. Color-code entities by domain (e.g., blue for fleet, green for dispatch, orange for user management).

5. User Flow Diagram

Purpose: Shows user navigation paths through the application. Used for UX planning, feature scoping, and identifying friction points.

5.1 Logistics Manager Flow

Entry: Login → MFA Verification

Primary Path: Dashboard → Truck Availability View → Filter/Sort Trucks → Select Truck → Create Dispatch → Confirm Assignment → Monitor Active Trip → View Delivery Confirmation

Secondary Paths:

- Dashboard → Analytics → Utilization Report → Export PDF
- Dashboard → Partner Management → Review Partner Performance → Adjust Reliability Score
- Dashboard → Settings → Notification Preferences → Save

Exit: Logout

5.2 Dispatcher Flow

Entry: Login

Primary Path: Daily Dispatch Board → Review Pending Requests → Match Truck to Load → Assign Driver → Send Notification → Monitor Progress → Handle Exception (if delay) → Confirm Completion

Secondary Paths:

- Daily Dispatch Board → Search Historical Dispatches → Duplicate Previous Dispatch
- Active Trips Map → Click Truck → View GPS History → Share Location with Customer

Exit: Logout

5.3 Driver Mobile App Flow

Entry: Biometric Login / PIN

Primary Path: Job List → Accept Assignment → View Trip Details → Start Navigation → Arrive at Pickup → Confirm Loading → Begin Transit → GPS Auto-Tracking → Arrive at Destination → Confirm Delivery → Capture Proof (Photo/Signature/QR) → Submit → Return to Available Status

Exception Paths:

- In Transit → Report Delay → Select Reason → Notify Dispatcher
- In Transit → Request Assistance → Emergency Contact / Breakdown Report
- Job List → Decline Assignment → Select Reason → System Reassigns

Exit: End Shift / Background Mode

Best Practice: Use swimlane diagrams for multi-role flows. Highlight decision points (diamonds) and error paths (red lines). Include time estimates for critical paths.

6. Business Process Flow Diagram

Purpose: Shows operational workflows from business perspective. Used for workflow optimization, SOP documentation, and identifying automation opportunities.

6.1 Standard Dispatch Process

Start: Cargo Ready for Shipment

Step 1: Check Truck Availability → Query platform for available trucks matching cargo specs

Step 2: Select Optimal Truck → Consider proximity, capacity, partner reliability, driver availability

Step 3: Assign Truck & Driver → Create dispatch record, lock truck availability

Step 4: Notify Driver → Push notification + SMS fallback with trip details

Step 5: Driver Acknowledges → Confirm assignment within timeout window

Step 6: Load Cargo → Driver updates status to 'Loading', warehouse confirms

Step 7: Transit → GPS tracking active, automated ETA updates, exception monitoring

Step 8: Arrive at Destination → Geofence trigger, status auto-updates to 'Arrived'

Step 9: Deliver Cargo → Recipient confirmation, proof of delivery capture

Step 10: Return to Base → Driver updates to 'Returning', truck becomes available

End: Dispatch Completed → Analytics recorded, partner performance updated

6.2 Exception Handling Sub-Processes

Driver Rejection: Driver declines assignment → System notifies dispatcher → Dispatcher selects alternate driver → Repeat from Step 3

Breakdown: Truck breakdown reported → Status changes to 'Maintenance' → Dispatcher reassigns load → Maintenance record created

Delay: ETA exceeds threshold → Automated alert to dispatcher and customer → Dispatcher assesses impact → Optional rerouting or alternate truck dispatch

Proof of Delivery Dispute: Customer disputes delivery → Audit trail review → Photo/signature evidence presented → Resolution logged

Best Practice: Use BPMN 2.0 notation for formal process documentation. Include lane separators for different roles (Dispatcher, Driver, Warehouse, System). Mark automated steps with gear icons.

7. Sequence Diagram

Purpose: Shows interactions between actors and system components over time. Used for API design, real-time system planning, and understanding temporal dependencies.

7.1 Dispatch Assignment Sequence

Actors: Dispatcher, Web Dashboard, API Server, Dispatch Engine, Database, Notification Service, Driver Mobile App

Sequence:

1. Dispatcher → Web Dashboard: "Create dispatch request" (origin, destination, cargo, priority)
2. Web Dashboard → API Server: POST /api/dispatches (JWT token, request body)
3. API Server → Middleware: Validate token, check permissions, resolve tenant
4. API Server → Dispatch Engine: "Process dispatch request"
5. Dispatch Engine → Database: Query available trucks (status='available', capacity >= required, location near origin)
6. Database → Dispatch Engine: Return candidate trucks list
7. Dispatch Engine → Dispatch Engine: Score candidates (proximity, reliability, capacity fit)
8. Dispatch Engine → Database: Lock truck status (optimistic locking, transaction)
9. Database → Dispatch Engine: Confirm lock, return truck record
10. Dispatch Engine → Database: Create dispatch record (status='pending')
11. Database → Dispatch Engine: Confirm creation, return dispatch ID
12. Dispatch Engine → Notification Service: Queue driver notification job
13. Notification Service → Driver Mobile App: Push notification (dispatch details, accept/decline)
14. Driver Mobile App → API Server: PATCH /api/dispatches/{id}/accept
15. API Server → Database: Update dispatch status to 'assigned'
16. Database → API Server: Confirm update
17. API Server → Web Dashboard: WebSocket event 'dispatch.assigned'
18. Web Dashboard → Dispatcher: Real-time dashboard update with assigned status

Alt [Driver Declines]:

- 14a. Driver Mobile App → API Server: PATCH /api/dispatches/{id}/decline
- 15a. API Server → Dispatch Engine: Trigger re-assignment workflow
- 16a. Dispatch Engine → Database: Release truck lock, find next candidate
- 17a. Dispatch Engine → Notification Service: Queue notification to dispatcher
- 18a. Notification Service → Web Dashboard: Alert dispatcher of decline, suggest alternatives

7.2 GPS Tracking Sequence

Actors: Driver Mobile App, Real-Time Gateway, Tracking Service, Database, Analytics Engine, Web Dashboard

Sequence (repeating every 30 seconds during transit):

1. Driver Mobile App → Real-Time Gateway: WebSocket emit 'gps.update' (lat, lng, accuracy, speed,
-

timestamp)

2. Real-Time Gateway → Tracking Service: Validate GPS data (sanity checks, route corridor)
3. Tracking Service → Database: INSERT tracking_log record
4. Tracking Service → Real-Time Gateway: Broadcast 'gps.updated' to subscribed dashboards
5. Real-Time Gateway → Web Dashboard: WebSocket event with updated position
6. Tracking Service → Analytics Engine: Stream event to Kafka for time-series storage
7. Analytics Engine → Database: Aggregate into route progress metrics
8. Tracking Service → Tracking Service: Check geofence triggers (arrival, departure, deviation)
9. [If geofence triggered] Tracking Service → Notification Service: Queue geofence alert
10. [If deviation detected] Tracking Service → Notification Service: Queue route deviation alert

Best Practice: Use PlantUML or Mermaid for text-based sequence diagrams that version control well. Include activation bars (rectangles) to show when components are actively processing. Use 'alt' and 'opt' frames for conditional flows.

8. Deployment Diagram

Purpose: Shows how software artifacts are deployed to infrastructure nodes. Used for DevOps planning, capacity planning, and disaster recovery design.

8.1 Production Deployment Architecture

Layer 1: Edge / CDN

- Cloudflare / AWS CloudFront — DDoS protection, SSL termination, static asset caching, geographic distribution
- WAF rules for SQL injection, XSS, and bot mitigation

Layer 2: Load Balancing

- NGINX (x2, active-passive) — Reverse proxy, rate limiting, request routing, health checks
- AWS ALB / GCP Load Balancer — Auto-scaling group orchestration, SSL passthrough

Layer 3: Application Tier (Kubernetes Cluster)

- API Server Pods (min: 3, max: 20, HPA based on CPU/memory) — NestJS application containers
- Real-Time Gateway Pods (min: 2, max: 10, HPA based on WebSocket connections) — Socket.IO servers
- Analytics Worker Pods (min: 2, max: 8, CronJob + queue-based scaling) — Python batch processors
- Notification Worker Pods (min: 2, max: 6, queue depth scaling) — Bull queue consumers

Layer 4: Data Tier

- PostgreSQL Primary (x1, r6g.2xlarge) — Write operations, transactional consistency
- PostgreSQL Read Replicas (x2, r6g.xlarge) — Dashboard queries, analytics ETL
- Redis Cluster (6 nodes, 3 masters + 3 replicas) — Caching, sessions, pub/sub, queues
- ClickHouse (x3, distributed) — Time-series analytics, GPS logs, operational metrics
- S3 / MinIO (x3, replicated) — Object storage, backups, static assets

Layer 5: External Services

- Google Maps API (external) — Geocoding, routing, distance matrix
- Twilio (external) — SMS gateway
- SendGrid (external) — Email delivery
- Firebase Cloud Messaging (external) — Push notifications
- Kafka Cluster (managed, x3 brokers) — Event streaming backbone

8.2 Development / Staging Environments

Development: Docker Compose on local machines. Single PostgreSQL, single Redis, mocked external services.

Staging: Single Kubernetes namespace on shared cluster. Smaller instance types, production-like data

(anonymized snapshot), full external service integration.

Production: Multi-AZ Kubernetes cluster with dedicated node pools, managed database services, and 99.9% SLA.

Best Practice: Use UML deployment notation (3D boxes for nodes, rectangles for artifacts). Include redundancy indicators and network segmentation (DMZ, internal, database subnets). Document backup and failover procedures alongside the diagram.

9. Data Flow Diagram

Purpose: Shows movement of data through the system. Used for analytics design, GDPR/privacy compliance, and understanding data transformation pipelines.

9.1 GPS Data Flow Pipeline

Source: Driver Mobile App (GPS sensor, 30-second intervals)

Flow 1 — Real-Time Path:

Mobile App → WebSocket → Real-Time Gateway → Validation Layer → Redis Pub/Sub → Dashboard Subscribers (live map update, <500ms latency)

Flow 2 — Persistence Path:

Validation Layer → Kafka Topic 'gps.raw' → Stream Processor (Flink/ksqlDB) → Enrichment (add route segment, speed classification) → Kafka Topic 'gps.enriched' → ClickHouse (time-series storage, 90-day hot, 2-year cold S3)

Flow 3 — Analytics Path:

ClickHouse → Scheduled ETL (hourly) → PostgreSQL Analytics Schema → Dashboard Queries (aggregated metrics, trend analysis)

Flow 4 — Alerting Path:

Stream Processor → Anomaly Detection (deviation, stoppage, speed violation) → Kafka Topic 'alerts' → Notification Service → Multi-channel alert dispatch

Flow 5 — Audit Path:

Validation Layer → Audit Log Buffer → PostgreSQL Audit Table (append-only, encrypted, 7-year retention)

9.2 Dispatch Data Flow

Source: Dispatcher / Logistics Manager (Web Dashboard)

Flow:

Dashboard → REST API → Validation → Business Logic → PostgreSQL (transactional record) → Event Bus → Notification Service (driver alert) → Real-Time Gateway (dashboard update) → Analytics Engine (KPI update)

Data Transformations:

- Raw request → Validated DTO → Domain entity → Database record
 - Database change event → CDC stream → Search index update (Elasticsearch)
 - Status transition → Audit log entry → Compliance report aggregation
-

Best Practice: Use Gane-Sarson or Yourdon/DeMarco notation. Label data stores with D numbers, processes with P numbers, and external entities with E numbers. Include data volume estimates (events/second) on flow lines.

10. Event Architecture Diagram

Purpose: Shows event-driven communication patterns between services. Used for scalability design, decoupling strategies, and understanding eventual consistency boundaries.

10.1 Core Event Topology

The platform uses Apache Kafka as the central event backbone with the following topic topology:

- **truck.status.changed**

Produced by: Driver App, Partner Portal. Consumed by: Real-Time Gateway, Analytics Engine, Notification Service. Payload: {truck_id, old_status, new_status, timestamp, location}

- **dispatch.created**

Produced by: API Server (Dispatch Component). Consumed by: Notification Service, Analytics Engine, Search Indexer. Payload: {dispatch_id, truck_id, driver_id, origin, destination, priority}

- **dispatch.assigned**

Produced by: API Server. Consumed by: Real-Time Gateway, Notification Service, ERP Integration. Payload: {dispatch_id, truck_id, driver_id, assigned_at, estimated_duration}

- **dispatch.completed**

Produced by: API Server. Consumed by: Analytics Engine, Billing Service, Partner Scoring. Payload: {dispatch_id, actual_duration, delay_minutes, proof_quality}

- **gps.location.updated**

Produced by: Real-Time Gateway. Consumed by: Tracking Service, Geofence Engine, Analytics Engine. Payload: {truck_id, lat, lng, accuracy, speed, timestamp}

- **delivery.proof.uploaded**

Produced by: API Server. Consumed by: Document Service, Compliance Auditor, Customer Notification. Payload: {delivery_id, proof_type, s3_url, timestamp}

- **partner.performance.updated**

Produced by: Analytics Engine. Consumed by: Partner Portal, Admin Panel. Payload: {partner_id, reliability_score, on_time_rate, period}

- **notification.sent**

Produced by: Notification Service. Consumed by: Delivery Tracker, Analytics Engine. Payload: {notification_id, user_id, channel, status, delivered_at}

- **user.login**

Produced by: Auth Component. Consumed by: Security Monitor, Audit Logger. Payload: {user_id, ip_address, user_agent, success, mfa_used}

- **maintenance.due**

Produced by: Maintenance Scheduler. Consumed by: Partner Portal, Notification Service. Payload: {truck_id, maintenance_type, due_date, estimated_downtime}

10.2 Event Processing Patterns

Publish-Subscribe: Real-time dashboard updates use pub/sub pattern — multiple subscribers receive the same event (dashboard, mobile app, analytics).

Competing Consumers: Notification processing uses competing consumers — multiple worker instances process from the same queue for horizontal scaling.

Event Sourcing: Critical state changes (dispatch status, truck status) are stored as immutable event streams, enabling state reconstruction and audit.

Saga Pattern: Multi-step operations (dispatch assignment) use saga orchestration — each step emits events, with compensating transactions for rollback on failure.

Best Practice: Use event storming notation (colored sticky notes: orange for events, blue for commands, yellow for aggregates, green for read models). Document event schemas with Avro or JSON Schema for backward compatibility.

11. Network Architecture Diagram

Purpose: Shows network topology, segmentation, and traffic flows. Critical for real-time systems, security hardening, and compliance audits.

11.1 Network Segmentation

Public Subnet (DMZ):

- Cloudflare / CDN endpoints
- NGINX load balancers (public-facing)
- NAT Gateway for outbound internet access

Security: Restricted inbound (443 only), full outbound. WAF rules active.

Application Subnet (Private):

- Kubernetes worker nodes
- API Server pods
- Real-Time Gateway pods
- Background worker pods

Security: No direct inbound from internet. Ingress only from DMZ via ALB. Egress to database and cache subnets only.

Data Subnet (Isolated):

- PostgreSQL primary and replicas
- Redis cluster nodes
- ClickHouse nodes
- Kafka brokers

Security: No internet access. Inbound only from application subnet on required ports (5432, 6379, 8123, 9092). Encryption at rest (AES-256) and in transit (TLS 1.3).

Management Subnet (Bastion):

- Jump host / bastion server
- Monitoring stack (Prometheus, Grafana)
- CI/CD runners

Security: VPN-only access. MFA required. Session recording enabled.

11.2 Real-Time Traffic Flows

Driver Mobile App → Real-Time Gateway:

Path: Mobile (4G/5G/WiFi) → Internet → Cloudflare → NGINX ALB → Kubernetes Ingress → Real-Time Gateway Pod

Protocol: WebSocket over HTTPS (wss://)

Latency Target: <100ms round-trip

Bandwidth: ~1 KB per GPS update (30-second intervals)

Real-Time Gateway → Dashboard:

Path: Gateway Pod → Redis Pub/Sub → Other Gateway Pods → WebSocket → Dashboard Client

Protocol: WebSocket (wss://)

Latency Target: <500ms from GPS reception to dashboard render

Fan-out: 1 GPS event → N dashboard subscribers (N = active viewers for that truck)

API Server → Database:

Path: API Pod → Kubernetes CNI → Database Subnet → PostgreSQL

Protocol: PostgreSQL wire protocol over TLS

Connection Pool: PgBouncer (transaction mode), max 100 connections per pod

Timeout: 30s statement timeout, 5s connection timeout

Best Practice: Use network diagram symbols (cloud for internet, firewall for security groups, switch for load balancers). Include port numbers, protocol labels, and latency requirements on connection lines. Color-code by security zone (green = public, yellow = private, red = restricted).

12. Security Architecture Diagram

Purpose: Shows security controls, authentication flows, and data protection mechanisms. Required for security audits, compliance certifications, and enterprise sales.

12.1 Authentication Flow

Step 1: User → Web Dashboard: Enter credentials (email + password)

Step 2: Dashboard → API Gateway: POST /auth/login (HTTPS, TLS 1.3)

Step 3: API Gateway → Rate Limiter: Check IP and user rate limits (Redis counter)

Step 4: API Gateway → Auth Component: Validate credentials (bcrypt hash comparison)

Step 5: Auth Component → Database: Query user record (row-level security enforced)

Step 6: [If MFA enabled] Auth Component → MFA Service: Generate TOTP challenge → SMS/Authenticator App

Step 7: [If MFA valid] Auth Component → Token Service: Generate JWT (RS256, 15-min access + 7-day refresh)

Step 8: Token Service → API Gateway: Return tokens + user profile + permissions

Step 9: API Gateway → Dashboard: Set HTTP-only secure cookies + return tokens

Step 10: Dashboard → Local Storage: Store tokens (access in memory, refresh in secure cookie)

Subsequent Requests: Dashboard → API Gateway: Bearer token in Authorization header → JWT validation (signature + expiry + issuer) → Tenant resolution → Permission check → Business logic

12.2 Authorization & Access Control

RBAC Layer:

- User assigned to Role (Super Admin, Logistics Manager, Dispatcher, Partner Admin, Driver)
- Role mapped to Permissions (resource + action pairs: trucks:read, dispatches:create, analytics:export)
- Permission check at API Gateway (early rejection) and Service Layer (defense in depth)

Multi-Tenant Isolation:

- Tenant ID extracted from JWT claim
- All database queries include company_id filter (enforced by query builder middleware)
- PostgreSQL Row-Level Security (RLS) policies as final defense layer
- Cross-tenant query attempts logged as security events and trigger alerts

API Security:

- Rate limiting: 1000 req/min standard, 5000 req/min enterprise (per user + per IP)
 - Input validation: JSON Schema validation, SQL injection prevention (parameterized queries), XSS output encoding
 - CORS: Whitelist-only origins, strict credential policies
 - CSRF: Double-submit cookie pattern for web clients, token-based for mobile
-

12.3 Data Protection

In Transit: TLS 1.3 for all communications. Certificate pinning on mobile apps. HSTS headers with 1-year max-age.

At Rest: AES-256 encryption for database volumes, backup storage, and object storage. Key management via AWS KMS / HashiCorp Vault with 90-day rotation.

In Use: Memory encryption for sensitive fields (password hashes, API keys) using platform-level protections.

Backup Security: Encrypted backups stored in geographically separate regions. 7-day point-in-time recovery. Annual disaster recovery drills.

12.4 Security Monitoring

SIEM Integration: Security events streamed to centralized SIEM (Splunk / Datadog Security)

Alert Triggers:

- Multiple failed login attempts (>5 in 10 minutes)
- Cross-tenant data access attempts
- Unusual API traffic patterns (volume, geographic origin)
- Privilege escalation attempts
- GPS spoofing detection (impossible location jumps)

Incident Response: Automated account lockouts, manual security team escalation, forensic log preservation

Best Practice: Use lock-and-key symbols for encryption, shield icons for firewalls/WAF, and person icons for authentication. Include trust boundaries (dashed lines) between security zones. Document compliance mappings (SOC 2, ISO 27001, GDPR) alongside controls.

13. Recommended Diagram Creation Order

Follow this phased approach to architecture documentation, aligning diagram creation with project milestones:

Phase 1: Foundation (Weeks 1-4)

1. **Business Process Flow Diagram** — Align stakeholders on operational workflows and identify automation opportunities
2. **System Context Diagram (C4 L1)** — Define system boundaries and external dependencies for project scoping
3. **User Flow Diagram** — Validate UX approach and feature completeness with product team
4. **Database ERD** — Establish data model foundation before API design begins
5. **Container Diagram (C4 L2)** — Technology selection and high-level component allocation

Phase 2: Implementation (Weeks 5-12)

6. **Component Diagram (C4 L3)** — Code organization and module interfaces for development team
7. **Sequence Diagram** — API contract definition and real-time interaction design
8. **Deployment Diagram** — Infrastructure planning, cost estimation, and DevOps pipeline design

Phase 3: Scaling & Hardening (Weeks 13-20)

9. **Data Flow Diagram** — Analytics pipeline design, GDPR compliance, and data retention planning
10. **Event Architecture Diagram** — Scalability design, service decoupling, and event schema governance
11. **Network Architecture Diagram** — Security zoning, real-time latency optimization, and compliance audit preparation
12. **Security Architecture Diagram** — Enterprise security hardening, penetration test preparation, and certification documentation

Maintenance: Update diagrams at each major release (quarterly). Version control diagrams alongside code (PlantUML/Mermaid in Git). Review diagrams during architecture review board meetings.

Document Conclusion: This diagrams guide provides a comprehensive reference for visualizing every architectural aspect of the logistics coordination platform. From stakeholder-facing context diagrams to developer-focused sequence diagrams and operations-focused deployment diagrams, each visualization serves a specific communication purpose. By following the recommended creation order and notation standards, teams can maintain architectural clarity throughout the system lifecycle — from initial design

through production scaling.